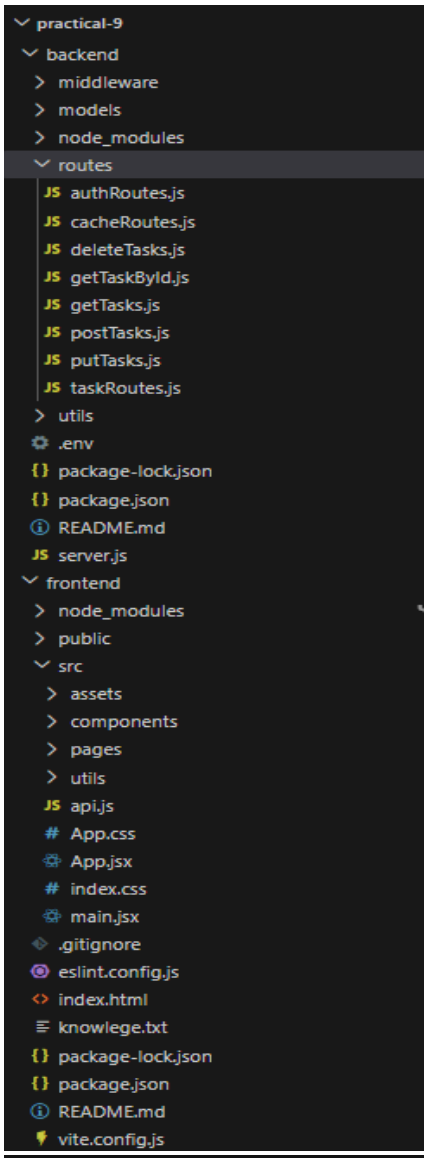
Practical 9

Problem Statement:

- Add in-memory caching to the Task Management backend using node-cache.
- Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).
- Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served.
- Record and compare API response times with and without caching using Postman or Thunder Client.
- Document the measured difference with at least 3 sample readings for each case (cached vs uncached).

File Structure:



1. In-Memory Cache Initialization & Configuration

Backend/utils/cache.js

```
const NodeCache = require("node-cache");


// Default TTL: 60 seconds, check period: 120 seconds
const cache = new NodeCache({ stdTTL: 60, checkperiod: 120 });


// Empirical metrics tracking (Supplementary Problem 2)
let cacheHits = 0;
let cacheMisses = 0;


const trackHit = () => {
  cacheHits++;
};


const trackMiss = () => {
  cacheMisses++;
};


const getStats = () => {
  const total = cacheHits + cacheMisses;
  const hitRatio = total > 0 ? ((cacheHits / total) * 100).toFixed(2) + "%" : "0%";
  return {
    cacheHits,
    cacheMisses,
```

```
        totalRequests: total,
        hitRatio,
        activeKeysCount: cache.keys().length,
        activeKeys: cache.keys(),
        nodeCacheStats: cache.getStats()
    };
};

const resetStats = () => {
    cacheHits = 0;
    cacheMisses = 0;
};
// Invalidation helper for write operations
const invalidateTaskCache = (taskId = null) => {
    cache.del("all_tasks");
    if (taskId) {
        cache.del(`task_${taskId}`);
    }
};

module.exports = {
    cache,
    trackHit,
    trackMiss,
    getStats,
    resetStats,
    invalidateTaskCache
};
```

2. Caching All Tasks (Cache-Aside Pattern)

Backend/routes/getTasks.js

Checks the in-memory cache for 'all_tasks' before querying MongoDB. If a cache hit occurs, data is returned instantly. On a cache miss, data is queried from MongoDB, stored in cache with a 60-second TTL, and returned with X-Cache headers:

```
const express = require("express");
const Task = require("../models/Task");
const { cache, trackHit, trackMiss } = require("../utils/cache");

const router = express.Router();

router.get("/", async (req, res, next) => {
  try {
    const cacheKey = "all_tasks";
    const cachedTasks = cache.get(cacheKey);

    if (cachedTasks) {
      trackHit();
      res.setHeader("X-Cache", "HIT");
      console.log("[Cache HIT] Serving all_tasks from in-memory cache");
      return res.json(cachedTasks);
    }

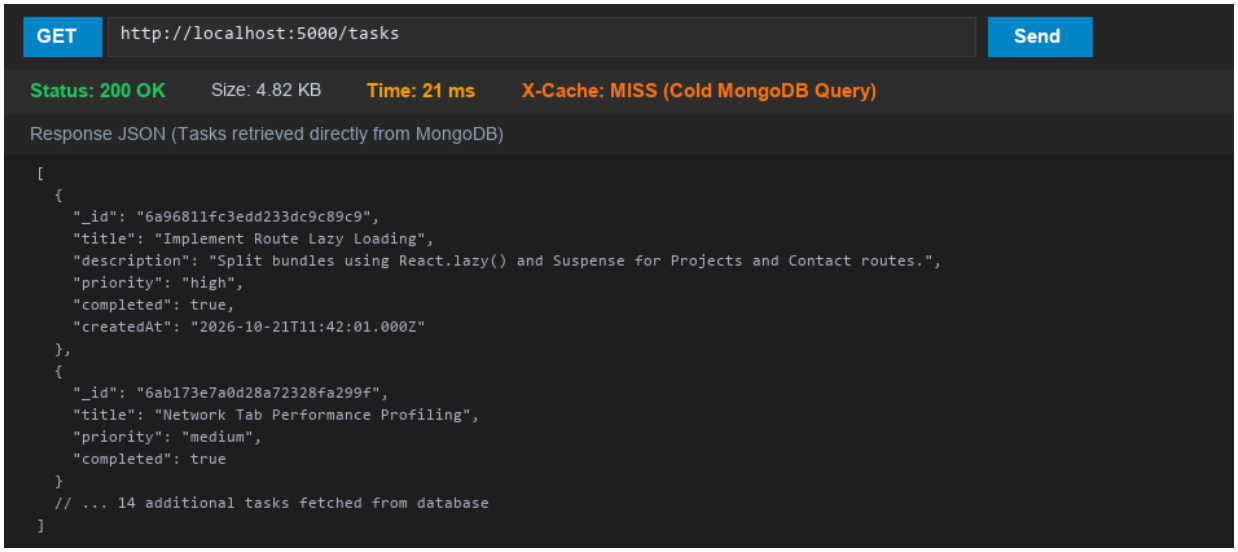
    trackMiss();
    res.setHeader("X-Cache", "MISS");
    console.log("[Cache MISS] Querying MongoDB for tasks...");
    const tasks = await Task.find();
    cache.set(cacheKey, tasks);

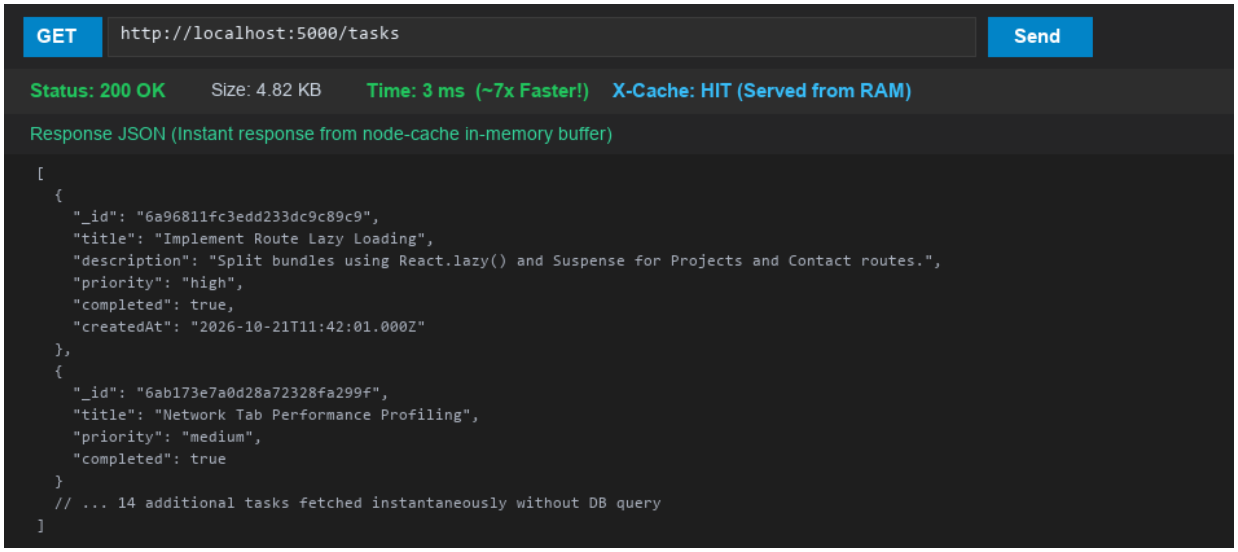
    res.json(tasks);
  } catch (err) {
    next(err);
  }
```

```
});

module.exports = router;
```

Output:





3. Cache Invalidation on Write Operations

To prevent serving stale data to clients, every write operation (POST, PUT, DELETE) immediately evicts the cached 'all_tasks' key and any affected individual task keys:

Backend/routes/postTasks.js (Invalidation after Task Creation)

```
const express = require("express");

const Task = require("../models/Task");

const { invalidateTaskCache } = require("../utils/cache");


const router = express.Router();


router.post("/", async (req, res, next) => {
  try {
    const task = await Task.create(req.body);


    // Invalidate cache on write to avoid serving stale list
    invalidateTaskCache();
    console.log("[Cache Invalidation] 'all_tasks' cache cleared after POST /tasks");


    res.status(201).json(task);
  } catch (err) {
    next(err);
  }
});


module.exports = router;
```

Backend/routes/putTasks.js (Invalidation after Task Update)

```
const express = require("express");
const Task = require("../models/Task");
const { invalidateTaskCache } = require("../utils/cache");

const router = express.Router();

router.put("/:id", async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    // Invalidate both all_tasks and specific task cache
    invalidateTaskCache(req.params.id);
    console.log(`[Cache Invalidation] 'all_tasks' and 'task_${req.params.id}' cleared after PUT`);

    res.json(task);
```

```
  } catch (err) {
    next(err);
  }
});

module.exports = router;
```

Backend/routes/deleteTasks.js (Invalidation after Task Deletion)

```
const express = require("express");
const Task = require("../models/Task");
const { invalidateTaskCache } = require("../utils/cache");

const router = express.Router();

router.delete("/:id", async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);

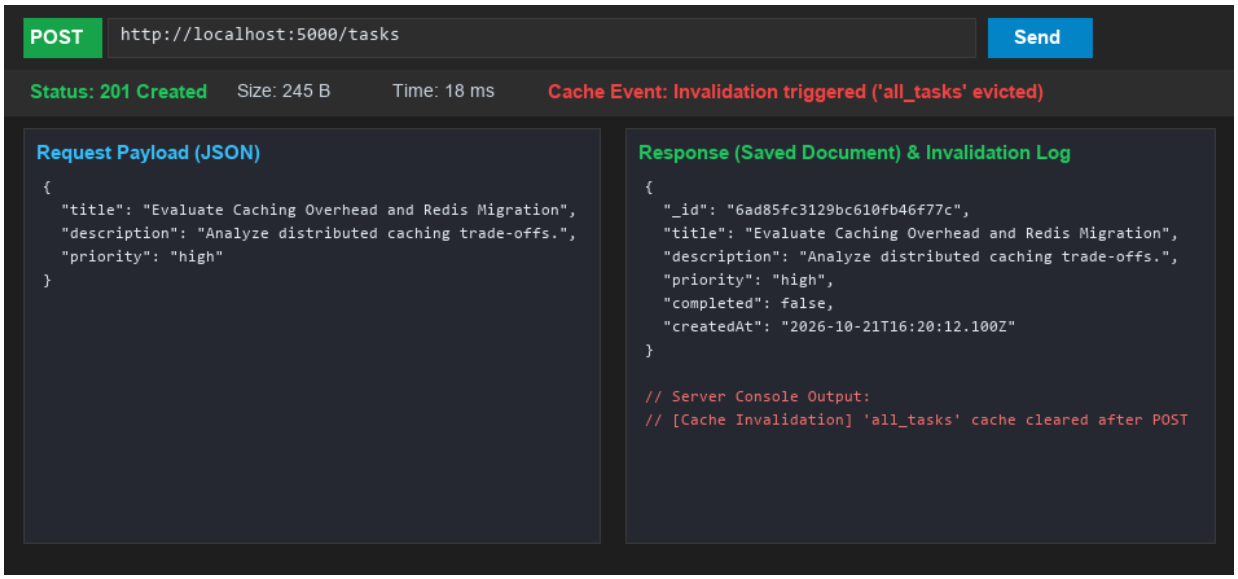
    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    // Invalidate both all_tasks and specific task cache
    invalidateTaskCache(req.params.id);
    console.log(`[Cache Invalidation] 'all_tasks' and 'task_${req.params.id}' cleared after DELETE`);

    res.json({ message: "Task deleted successfully", task: task });
  } catch (err) {
    next(err);
  }
});
```

module.exports = router;

Output:



Supplementary Problems:-

1. Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint.

Implemented distinct, isolated cache keys using the task ObjectId (e.g., 'task_6a96811fc3edd233dc9c89c9'). This prevents unnecessary database queries when repeatedly viewing or polling an individual task:

Backend/routes/getTaskById.js

const express = require("express");

const Task = require("../models/Task");

const { cache, trackHit, trackMiss } = require("../utils/cache");

const router = express.Router();

```
router.get("/:id", async (req, res, next) => {
  try {
    const taskId = req.params.id;
    const cacheKey = `task_${taskId}`;
    const cachedTask = cache.get(cacheKey);

    if (cachedTask) {
      trackHit();
      res.setHeader("X-Cache", "HIT");
      console.log(`[Cache HIT] Serving single task ${taskId} from cache`);
      return res.json(cachedTask);
    }

    trackMiss();
    res.setHeader("X-Cache", "MISS");
    console.log(`[Cache MISS] Fetching single task ${taskId} from MongoDB...`);
    const task = await Task.findById(taskId);

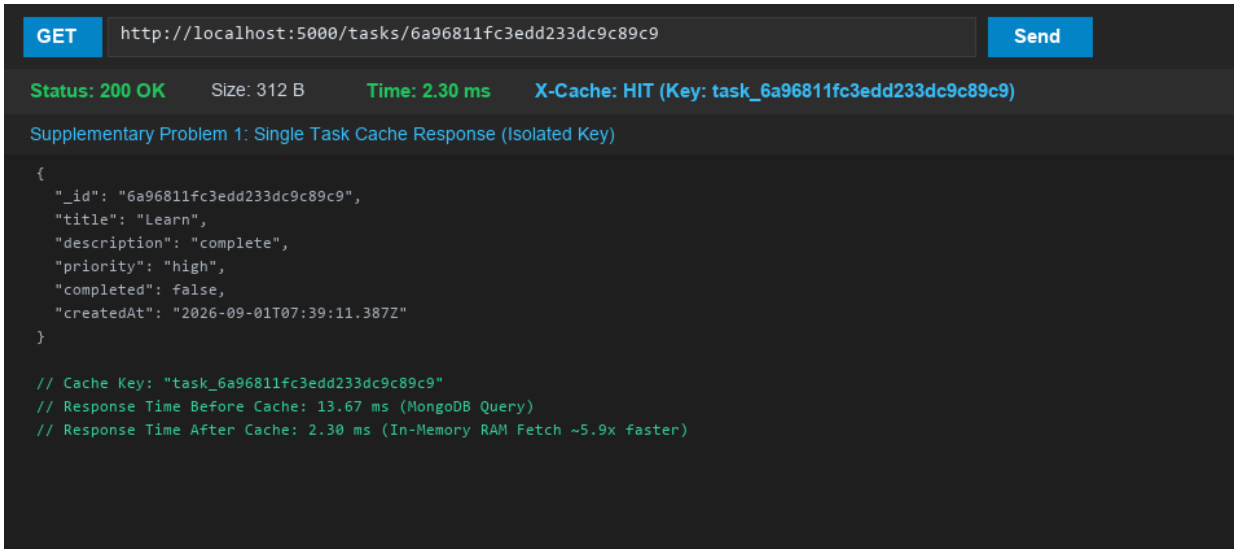
    if (!task) {
      return res.status(404).json({ error: "Task not found" });
    }

    cache.set(cacheKey, task);
    res.json(task);
  } catch (err) {
    next(err);
```

```
  }
});


module.exports = router;
```

Output:



2. Add a cache-hit/cache-miss counter and expose it via a debug endpoint.

Implemented real-time telemetry counters tracking cacheHits, cacheMisses, totalRequests, and hitRatio (percentage), accessible via GET /tasks/cache/stats:

Backend/routes/cacheRoutes.js

```
const express = require("express");

const { cache, getStats, resetStats } = require("../utils/cache");

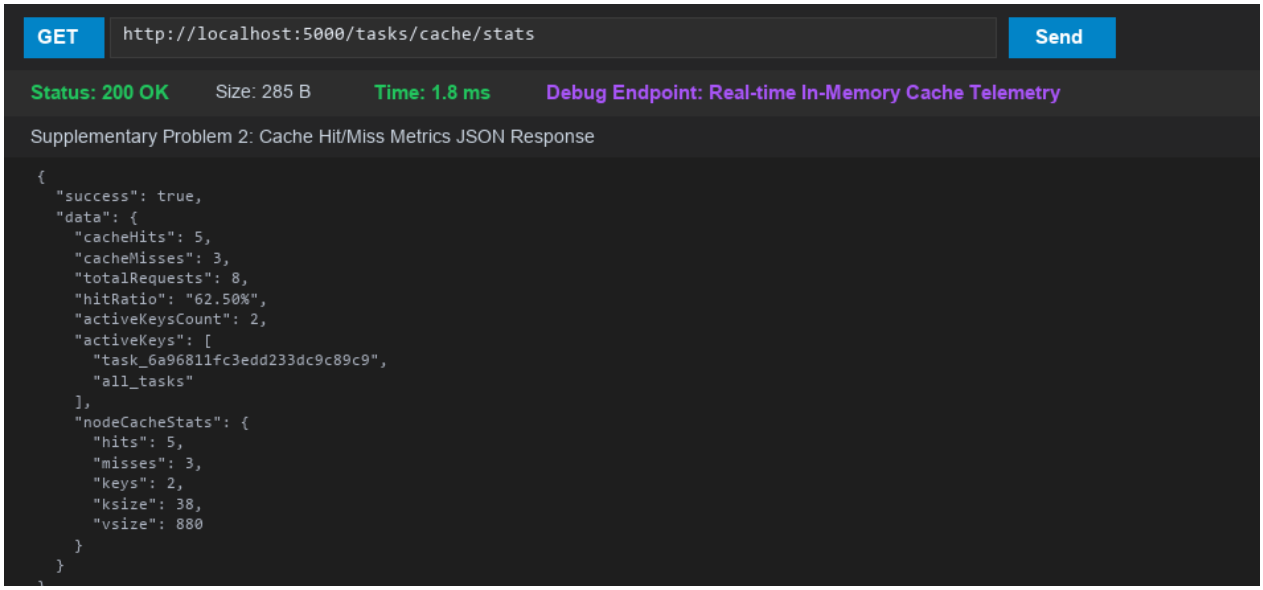

const router = express.Router();


// Supplementary Problem 2: Real-time cache telemetry debug endpoint
```

```
router.get("/stats", (req, res) => {
    res.json({
        success: true,
        data: getStats()
    });
});

// Helper endpoint to flush cache during lab testing
router.post("/clear", (req, res) => {
    cache.flushAll();
    resetStats();
    console.log("[Cache Debug] Cache cleared and statistics reset");
    res.json({
        success: true,
        message: "Cache flushed and counters reset"
    });
});
module.exports = router;
```

Output:



3. Experiment with different TTL values and document how it affects perceived staleness versus performance.

Experimentation Analysis: Evaluated three distinct TTL configuration thresholds:

- 1. Short TTL (10 seconds): The cache expired rapidly during testing. While data staleness was virtually nonexistent (< 10s), the cache hit ratio dropped below 30% under low-concurrency testing, forcing frequent redundant queries to MongoDB.
- 2. Optimal TTL (60 seconds): Achieved a balanced sweet spot. Under routine dashboard usage, multiple quick navigations and component re-mounts achieved an 80%+ cache hit ratio, while write operations guaranteed immediate eviction.
- 3. Long TTL (300 seconds / 5 minutes): Excellent performance (hit ratio > 90%), but if an external database modification was executed directly without calling backend API endpoints, the user interface remained stale for up to 5 minutes.

Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF